


# JAVA PROGRAMMING LAB (BCS-154)

## B.Tech CSE 1st YEAR

**SUBMITTED TO:**

**MR. ABHAY GUPTA SIR**

**SUBMITTED BY:**

**JATIN RATHOR**

**BCS2026462**

| SN. | PROGRAM NAME | PAGE | DATE | SIGNATURE |
|---|---|---|---|---|
| 1 | Hello World, Variables and Arithmetic | 1 | 26/10/2026 | |
| 2 | Decisions and Loops with Numbers | 4 | 26/10/2026 | |
| 3 | Star and Number Patterns | 7 | 26/10/2026 | |
| 4 | Writing Functions | 10 | 28/10/2026 | |
| 5 | Arrays: Traversal and Counting | 13 | 28/10/2026 | |
| 6 | Arrays: Reverse and Duplicates | 15 | 28/10/2026 | |
| 7 | Counting Steps and Complexity | 18 | 30/10/2026 | |
| 8 | Matrices: Transpose and Row Sums | 20 | 30/10/2026 | |
| 9 | Matrices: Rows and Columns | 23 | 30/10/2026 | |
| 10 | Switch and ArrayList | 26 | 02/11/2026 | |
| 11 | Bubble Sort and Selection Sort | 29 | 02/11/2026 | |
| 12 | Insertion Sort | 32 | 02/11/2026 | |
| 13 | Strings: Methods and Palindromes | 35 | 04/11/2026 | |
| 14 | Characters: Counting and Positions | 38 | 04/11/2026 | |
| 15 | StringBuilder Methods | 41 | 04/11/2026 | |

# EXPERIMENT NO. 1

**1. Experiment Title:**
Hello World, Variables and Arithmetic

**2. Aim:**
To print text, store values in variables and calculate with arithmetic operators.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
Every Java program starts execution from the main() method of a class. System.out.println() prints text and moves to the next line. A variable stores a value of a data type such as int, double, char or String. Arithmetic operators (+, -, *, /, %) calculate with numbers; for int values, / gives the whole-number quotient and % gives the remainder. The final keyword creates a constant whose value cannot change.

**5. Algorithm / Procedure:**
1. Start and create class Main with the main() method.
2. Declare variables (or a final constant) with their data types and assign the given values.
3. Calculate the required results using arithmetic operators.
4. Print every label followed by its value using System.out.println().
5. Compile with javac Main.java, run with java Main and note the output.

**6. Program / Queries / Commands:**

**Task 1: Hello World**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 1.1

public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println("Welcome to Java");
    }
}
```

**Task 2: Variables and data types**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 1.2

public class Main {
    public static void main(String[] args) {
        int roll = 101;
        double percentage = 84.5;
        char section = 'B';
        String name = "Asha";
        System.out.println("Roll: " + roll);
        System.out.println("Percentage: " + percentage);
        System.out.println("Section: " + section);
        System.out.println("Name: " + name);
    }
}
```

**Task 3: Arithmetic operators**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 1.3

public class Main {
    public static void main(String[] args) {
        int a = 17;
        int b = 5;
        System.out.println("Sum: " + (a + b));
        System.out.println("Difference: " + (a - b));
        System.out.println("Product: " + (a * b));
        System.out.println("Quotient: " + (a / b));
        System.out.println("Remainder: " + (a % b));
    }
}
```

**Task 4: Constant PI and circle**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 1.4

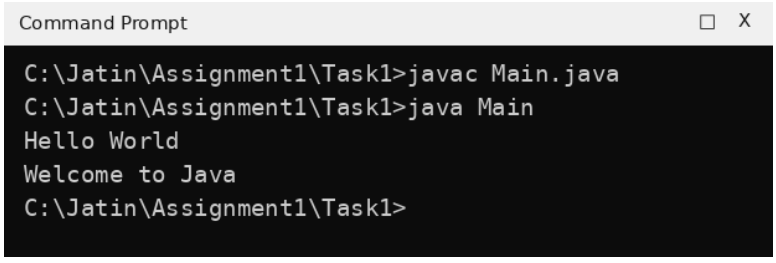
public class Main {
    public static void main(String[] args) {
        final double PI = 3.14;
        int radius = 7;
        double area = PI * radius * radius;
        double perimeter = 2 * PI * radius;
        System.out.println("Area: " + area);
        System.out.println("Perimeter: " + perimeter);
    }
}
```

**7. Input:**

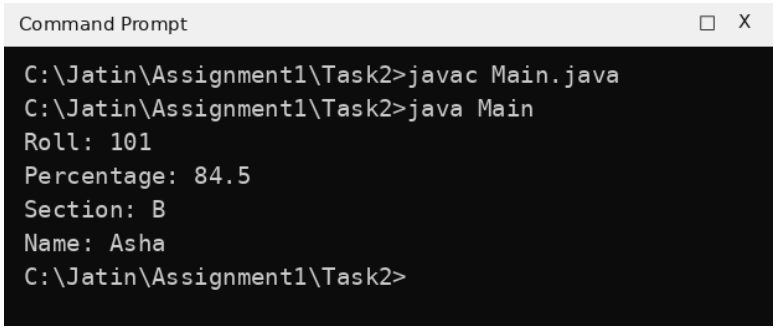
**Task 1:** No input required.
**Task 2:** roll = 101, percentage = 84.5, section = 'B', name = "Asha"
**Task 3:** a = 17, b = 5
**Task 4:** PI = 3.14, radius = 7

**8. Output:**

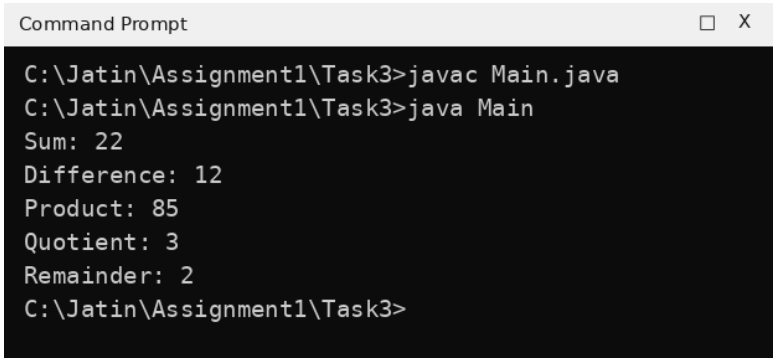
**Task 1: Hello World**



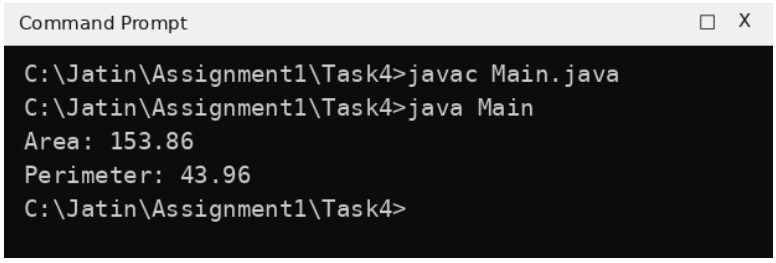
**Task 2: Variables and data types**




**Task 3: Arithmetic operators**




**Task 4: Constant PI and circle**




**9. Result:**

The programs for printing text, using variables, arithmetic operators and a final constant were written, compiled and run successfully. The output matched the expected output.

# EXPERIMENT NO. 2

**1. Experiment Title:**
Decisions and Loops with Numbers

**2. Aim:**
To use if, else if and loops to make decisions and repeat steps with whole numbers.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
The if-else statement chooses between two paths using a condition. An else-if ladder checks several conditions one after the other and runs the first one that is true. A for loop repeats a block a fixed number of times using an initialisation, a condition and an update. A number is even when n % 2 == 0, and a year is a leap year if it is divisible by 4 but not by 100, or divisible by 400.

**5. Algorithm / Procedure:**
1. Start and store the given number in a variable.
2. Evaluate the condition(s) with if / else-if, or run the for loop.
3. Print the decision or the calculated result.
4. Compile and run the program and note the output.

**6. Program / Queries / Commands:**

**Task 1: Even or odd**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 2.1

public class Main {
    public static void main(String[] args) {
        int n = 27;
        if (n % 2 == 0) {
            System.out.println(n + " is even");
        } else {
            System.out.println(n + " is odd");
        }
    }
}
```

**Task 2: Grade using else-if ladder**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 2.2

public class Main {
    public static void main(String[] args) {
        int marks = 83;
        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else {
            System.out.println("Grade: C");
        }
    }
}
```

**Task 3: Leap year**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 2.3

public class Main {
    public static void main(String[] args) {
        int year = 2024;
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
            System.out.println("Leap year");
        } else {
            System.out.println("Not a leap year");
        }
    }
}
```

**Task 4: Factorial of 5**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 2.4

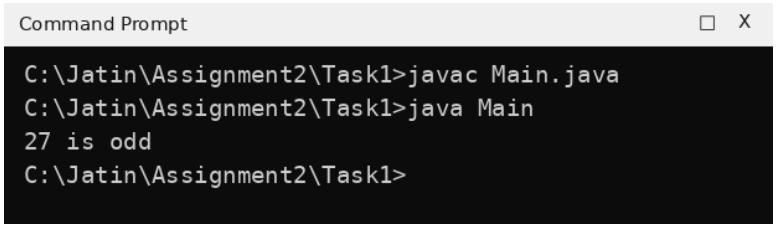
public class Main {
    public static void main(String[] args) {
        int fact = 1;
        for (int i = 1; i <= 5; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial of 5 is " + fact);
    }
}
```

**7. Input:**

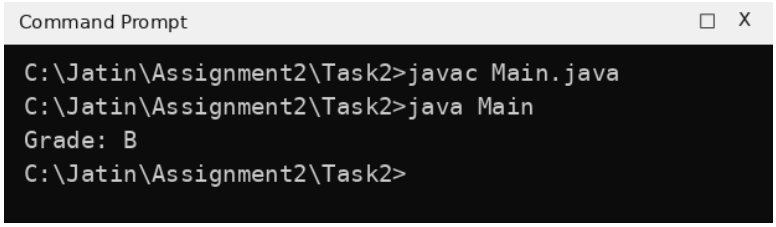
**Task 1:** n = 27
**Task 2:** marks = 83
**Task 3:** year = 2024
**Task 4:** Number = 5

**8. Output:**

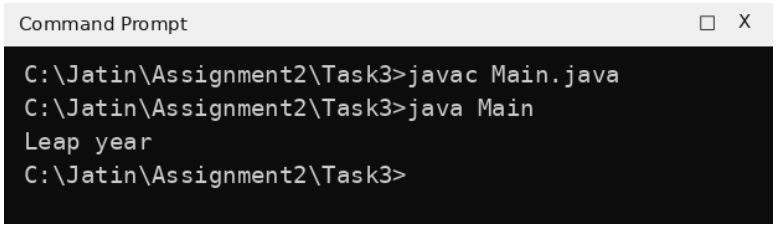
**Task 1: Even or odd**



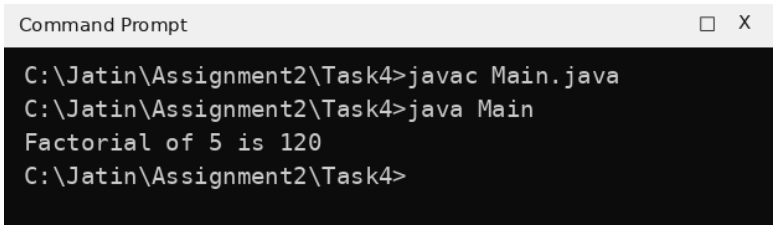

**Task 2: Grade using else-if ladder**



**Task 3: Leap year**




**Task 4: Factorial of 5**




**9. Result:**

Programs using if-else, else-if ladder, leap year condition and a for loop were executed successfully and produced the expected output.

# EXPERIMENT NO. 3

**1. Experiment Title:**
Star and Number Patterns

**2. Aim:**
To use a loop inside another loop to print stars and numbers in rows.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
A nested loop is a loop placed inside another loop. For every single pass of the outer loop, the inner loop runs completely. In pattern printing, the outer loop controls the rows and the inner loop controls what is printed in each row. System.out.print() prints without moving to a new line, and System.out.println() after the inner loop ends the row.

**5. Algorithm / Procedure:**
1. Use an outer loop for the rows.
2. Use an inner loop to print the stars or numbers of the current row with System.out.print().
3. After the inner loop, print a new line.
4. Compile and run the program and compare the pattern.

**6. Program / Queries / Commands:**

**Task 1: Square of stars**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 3.1

public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 4; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

**Task 2: Right triangle of stars**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 3.2

public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

**Task 3: Number triangle**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 3.3

public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

**Task 4: Upside-down triangle of stars**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 3.4

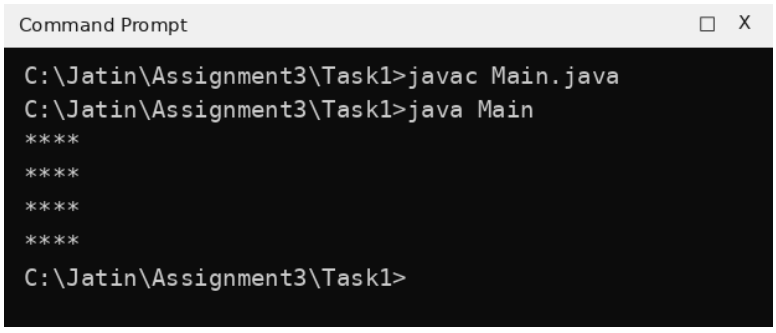
public class Main {
    public static void main(String[] args) {
        for (int i = 4; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

**7. Input:**

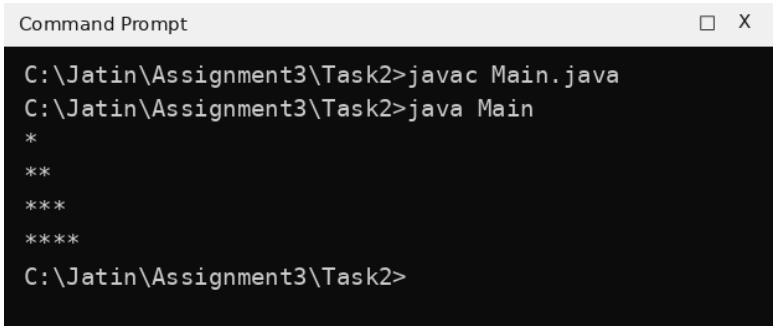
**Task 1:** Rows = 4, Columns = 4
**Task 2:** Rows = 4
**Task 3:** Rows = 4
**Task 4:** Rows = 4

**8. Output:**

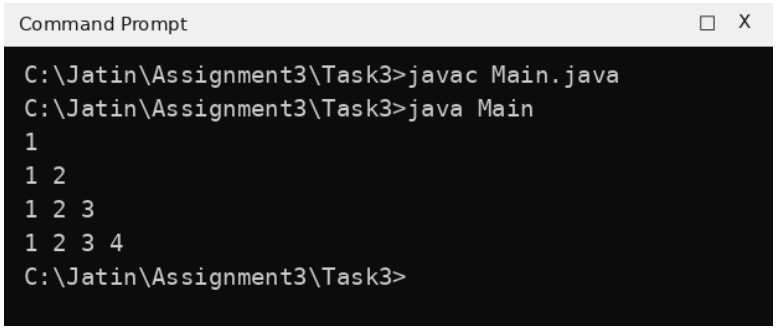
**Task 1: Square of stars**



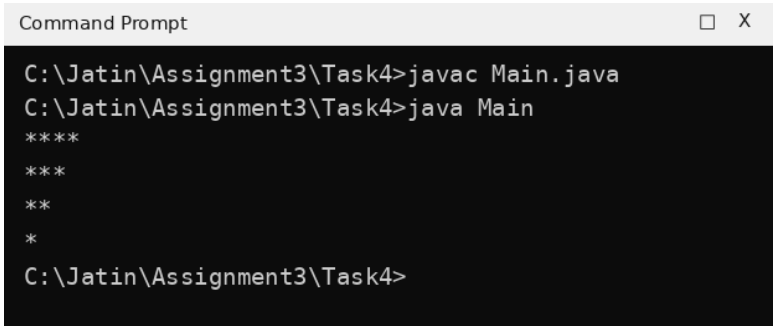
**Task 2: Right triangle of stars**




**Task 3: Number triangle**




**Task 4: Upside-down triangle of stars**




**9. Result:**

Square, right triangle, number triangle and inverted triangle patterns were printed successfully using nested loops.

# EXPERIMENT NO. 4

**1. Experiment Title:**
Writing Functions

**2. Aim:**
To write functions (methods) with parameters and return values, and call them from main.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
A function is a named block of code that can be called again and again. It can take parameters as input and return a value using the return statement. Functions make a program shorter and easier to understand, and one function can call another. nCr = n! / (r! x (n-r)!) and nPr = n! / (n-r)!.

**5. Algorithm / Procedure:**
1. Write the required function(s) with their parameters and return type.
2. Inside the function, calculate the result with a loop or an expression and return it.
3. Call the function from main() with the given values.
4. Print the returned value and note the output.

**6. Program / Queries / Commands:**

**Task 1: nCr using fact()**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 4.1

public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) {
            f = f * i;
        }
        return f;
    }

    static int nCr(int n, int r) {
        return fact(n) / (fact(r) * fact(n - r));
    }

    public static void main(String[] args) {
        System.out.println("5C2 = " + nCr(5, 2));
    }
}
```

**Task 2: nPr using fact()**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 4.2

public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) {
            f = f * i;
        }
        return f;
    }

    static int nPr(int n, int r) {
        return fact(n) / fact(n - r);
    }

    public static void main(String[] args) {
        System.out.println("5P2 = " + nPr(5, 2));
    }
}
```

**Task 3: Larger of two numbers**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 4.3

public class Main {
    static int maxOf(int a, int b) {
        if (a > b) {
            return a;
        }
        return b;
    }

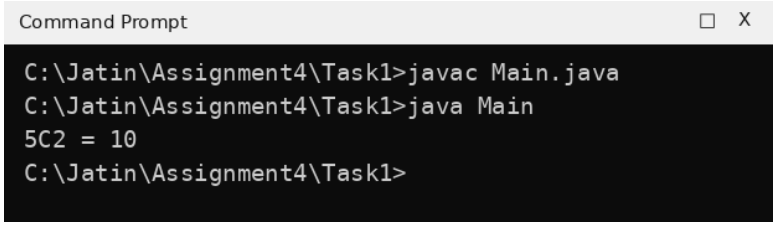
    public static void main(String[] args) {
        System.out.println("Larger: " + maxOf(14, 9));
    }
}
```

**7. Input:**

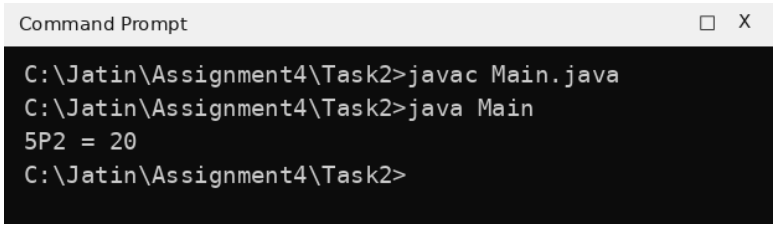
**Task 1:** n = 5, r = 2
**Task 2:** n = 5, r = 2
**Task 3:** a = 14, b = 9

**8. Output:**

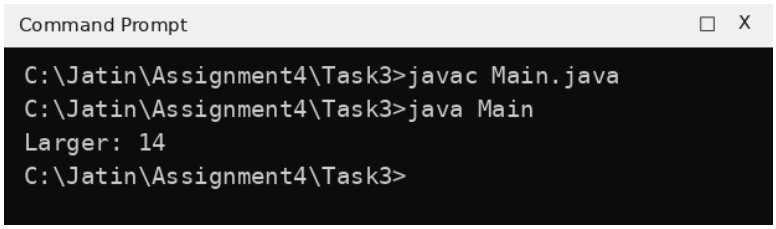
**Task 1: nCr using fact()**



**Task 2: nPr using fact()**




**Task 3: Larger of two numbers**




**9. Result:**

Functions for factorial, nCr, nPr and maximum were written and called from main() successfully, and the output matched the expected output.

# EXPERIMENT NO. 5

**1. Experiment Title:**
Arrays: Traversal and Counting

**2. Aim:**
To create an array, visit every element with a loop, and add up or count the values.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
An array stores many values of the same type in one variable. Elements are accessed by an index that starts from 0, and arr.length gives the number of elements. Traversal means visiting every element with a loop. While traversing, we can keep a running sum, track the largest value or count the elements that satisfy a condition.

**5. Algorithm / Procedure:**
1. Create the array with the given values.
2. Run a for loop from index 0 to arr.length - 1.
3. Print each element, or update the sum / largest / count variable inside the loop.
4. Print the final result after the loop.

**6. Program / Queries / Commands:**

**Task 1: Print index and element**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 5.1

public class Main {
    public static void main(String[] args) {
        int[] arr = {3, 4, 5};
        for (int i = 0; i < arr.length; i++) {
            System.out.println("Index " + i + ": " + arr[i]);
        }
    }
}
```

**Task 2: Sum and largest value**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 5.2

public class Main {
    public static void main(String[] args) {
        int[] arr = {12, 45, 7, 33, 21};
        int sum = 0;
        int largest = arr[0];
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
            if (arr[i] > largest) {
                largest = arr[i];
            }
        }
        System.out.println("Sum: " + sum);
        System.out.println("Largest: " + largest);
    }
}
```

**Task 3: Count values greater than 10**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 5.3

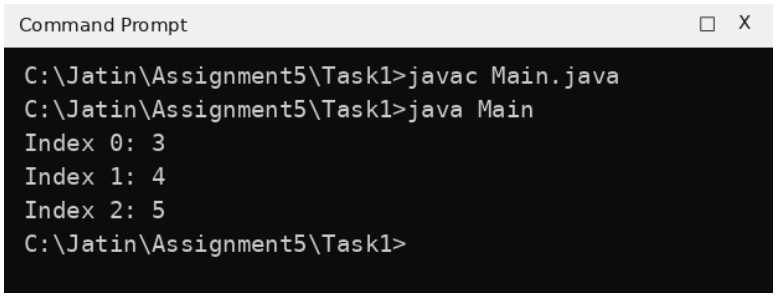
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 12, 2, 14, 7};
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > 10) {
                count++;
            }
        }
        System.out.println("Count greater than 10: " + count);
    }
}
```

**7. Input:**

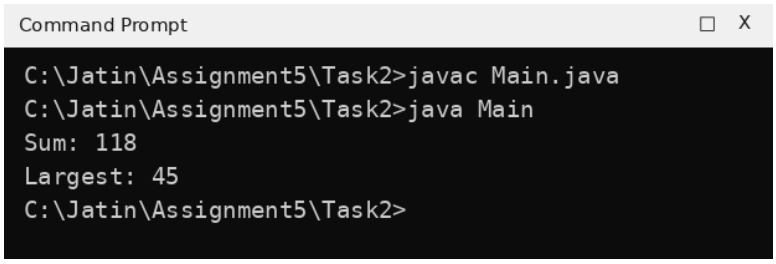
**Task 1:** arr = {3, 4, 5}
**Task 2:** arr = {12, 45, 7, 33, 21}
**Task 3:** arr = {5, 12, 2, 14, 7}

**8. Output:**

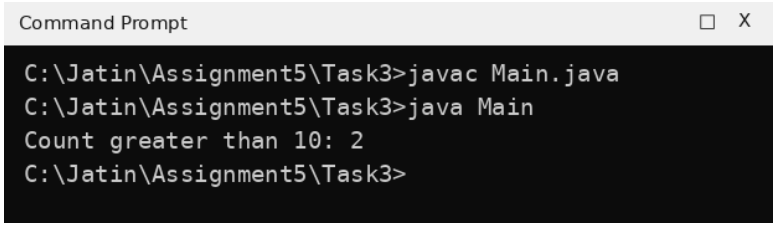
**Task 1: Print index and element**




**Task 2: Sum and largest value**




**Task 3: Count values greater than 10**




**9. Result:**

Arrays were created and traversed with loops to print elements, find the sum and largest value, and count values greater than 10. The output was correct.

# EXPERIMENT NO. 6

**1. Experiment Title:**
Arrays: Reverse and Duplicates

**2. Aim:**
To print an array in reverse order, find the smallest value, and check an array for repeated values.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
To print an array in reverse, a loop runs from the last index (length - 1) down to 0. To find the smallest value, assume the first element is smallest and update it whenever a smaller element is found, remembering its index. To find a repeated value, nested loops compare every element with all the elements after it.

**5. Algorithm / Procedure:**
1. Create the array with the given values.
2. For reverse: loop from arr.length - 1 down to 0 and print each element.
3. For smallest: store arr[0] as smallest, compare with every other element and update value and index.
4. For duplicates: compare arr[i] with arr[j] for every j > i using nested loops; stop when a match is found.
5. Print the result.

**6. Program / Queries / Commands:**

**Task 1: Print array in reverse**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 6.1

public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        for (int i = arr.length - 1; i >= 0; i--) {
            System.out.println(arr[i]);
        }
    }
}
```

**Task 2: Smallest value and its index**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 6.2

public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 5, 1, 8, 3, 4};
        int smallest = arr[0];
        int index = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < smallest) {
                smallest = arr[i];
                index = i;
            }
        }
        System.out.println("Smallest: " + smallest + " at index " + index);
    }
}
```

**Task 3: Find repeated value**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 6.3

public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 9, 2, 9, 7};
        boolean found = false;
        int repeated = 0;
        for (int i = 0; i < arr.length && !found; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    repeated = arr[i];
                    found = true;
                    break;
                }
            }
        }
        if (found) {
            System.out.println("Repeated value: " + repeated);
        } else {
            System.out.println("No repeated value");
        }
    }
}
```

**7. Input:**

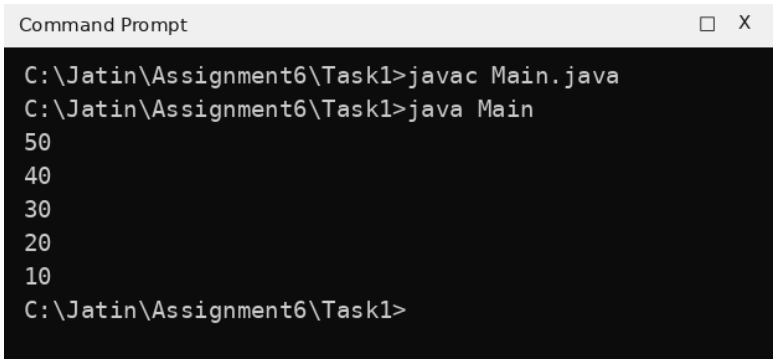
**Task 1:** arr = {10, 20, 30, 40, 50}

**Task 2:** arr = {2, 5, 1, 8, 3, 4}

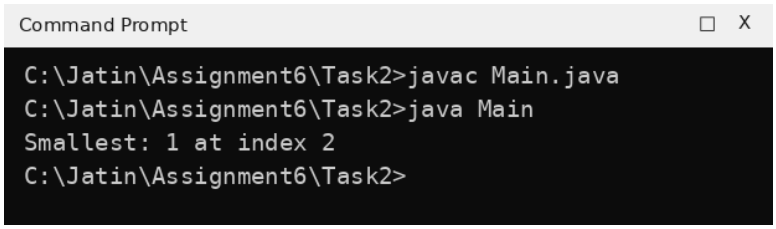
**Task 3:** arr = {4, 9, 2, 9, 7}

## 8. Output:

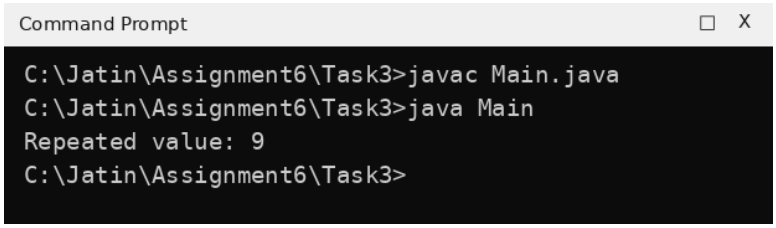
### Task 1: Print array in reverse




### Task 2: Smallest value and its index




### Task 3: Find repeated value




## 9. Result:

Reverse printing, smallest value with its index, and repeated value detection were implemented and verified successfully.

# EXPERIMENT NO. 7

**1. Experiment Title:**
Counting Steps and Complexity

**2. Aim:**
To count the steps a program takes, and write its time complexity as a comment.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
Time complexity tells how the number of steps grows with the input size n. A single loop that runs n times takes O(n) time. Two nested loops that both run n times take O(n^2) time. A loop that halves the value in every step runs about log2(n) times, so it takes O(log n) time.

**5. Algorithm / Procedure:**
1. Declare a counter variable steps and set it to 0.
2. Run the loop(s) and increase the counter once for every step.
3. Write the time complexity as a comment above the loop.
4. Print the counter and compare it with the expected number of steps.

**6. Program / Queries / Commands:**

**Task 1: Single loop - O(n)**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 7.1

public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n)
        for (int i = 1; i <= 10; i++) {
            steps++;
        }
        System.out.println("Steps: " + steps);
    }
}
```

**Task 2: Nested loops - O(n^2)**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 7.2

public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n^2)
        for (int i = 1; i <= 10; i++) {
            for (int j = 1; j <= 10; j++) {
                steps++;
            }
        }
        System.out.println("Steps: " + steps);
    }
}
```

**Task 3: Halving loop - O(log n)**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 7.3

public class Main {
    public static void main(String[] args) {
        int n = 1024;
        int steps = 0;
        // Time complexity: O(log n)
        while (n > 1) {
            n = n / 2;
            steps++;
        }
        System.out.println("Halving steps: " + steps);
    }
}
```

**7. Input:**

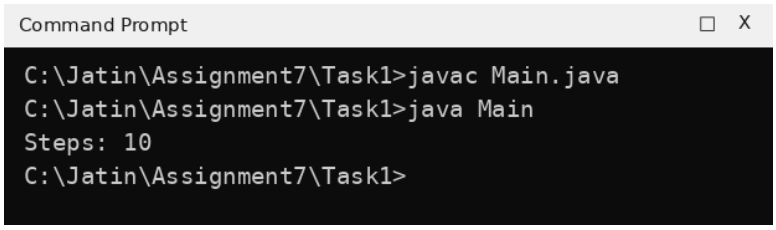
**Task 1:** Loop from 1 to 10

**Task 2:** Both loops from 1 to 10

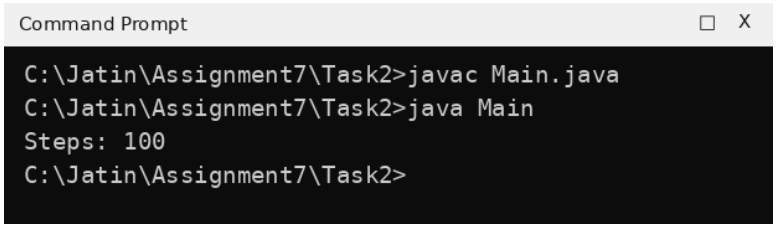
**Task 3:** n = 1024

**8. Output:**

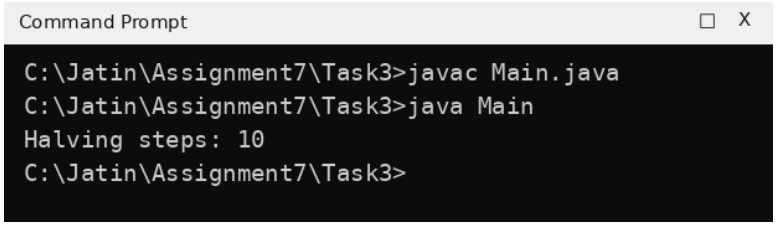
**Task 1: Single loop - O(n)**




**Task 2: Nested loops - O(n^2)**




**Task 3: Halving loop - O(log n)**




**9. Result:**

The number of steps was counted for a single loop (10), nested loops (100) and a halving loop (10), and their time complexities O(n), O(n^2) and O(log n) were written as comments.

# EXPERIMENT NO. 8

**1. Experiment Title:**
Matrices: Transpose and Row Sums

**2. Aim:**
To create two-dimensional arrays, visit them row by row, and build a new matrix from one.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
A two-dimensional array (matrix) is an array of arrays, declared as int[][] m = new int[rows][cols]. An element is accessed as m[i][j], where i is the row and j is the column. Nested loops visit the matrix row by row. The transpose of a matrix swaps its rows and columns, so t[j][i] = m[i][j]. The row sum is the total of all elements in one row.

**5. Algorithm / Procedure:**
1. Create the matrix (fill it with nested loops, or give the values directly).
2. Use the outer loop for rows and the inner loop for columns.
3. For transpose: copy m[i][j] into t[j][i]. For row sums: add all elements of a row in a variable and print it after the inner loop.
4. Print the result and note the output.

**6. Program / Queries / Commands:**

**Task 1: Fill 3x4 matrix with 1 to 12**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 8.1

public class Main {
    public static void main(String[] args) {
        int[][] m = new int[3][4];
        int value = 1;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                m[i][j] = value;
                value++;
            }
        }
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                System.out.print(m[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

**Task 2: Transpose of 3x3 matrix**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 8.2

public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int[][] t = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                t[j][i] = m[i][j];
            }
        }
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(t[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

**Task 3: Sum of each row**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 8.3

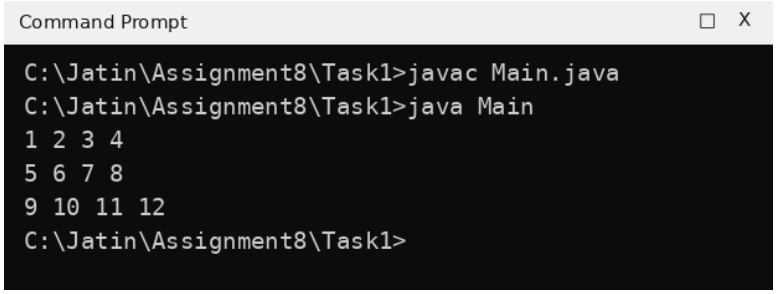
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        for (int i = 0; i < 3; i++) {
            int sum = 0;
            for (int j = 0; j < 3; j++) {
                sum = sum + m[i][j];
            }
            System.out.println("Row " + (i + 1) + " sum: " + sum);
        }
    }
}
```

**7. Input:**

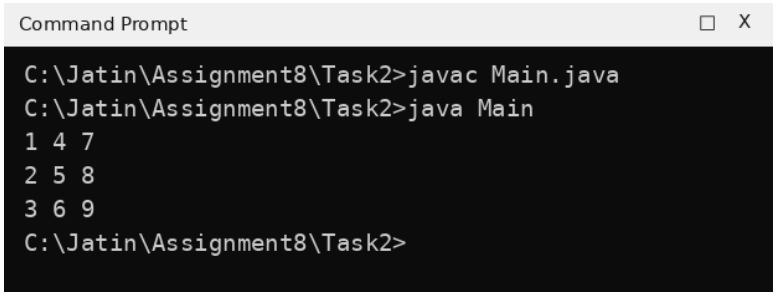
**Task 1:** 3 rows, 4 columns; values 1 to 12
**Task 2:** Matrix rows: 1 2 3, 4 5 6, 7 8 9
**Task 3:** Matrix rows: 1 2 3, 4 5 6, 7 8 9

**8. Output:**

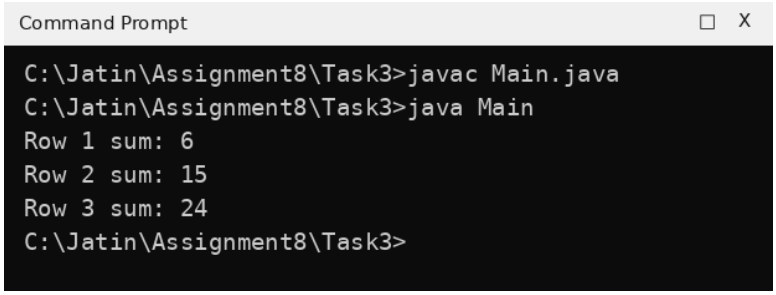
**Task 1: Fill 3x4 matrix with 1 to 12**



**Task 2: Transpose of 3x3 matrix**




**Task 3: Sum of each row**




**9. Result:**

Matrix creation, transpose and row sums were performed using nested loops and the output matched the expected output.

# EXPERIMENT NO. 9

**1. Experiment Title:**
Matrices: Rows and Columns

**2. Aim:**
To visit the rows and columns of a matrix with nested loops to print, add and count its values.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
In a matrix m[i][j], the first index is the row and the second index is the column. The first row is m[0] and the last row is m[m.length - 1]. To add a column, the outer loop runs over the columns and the inner loop runs over the rows. A counter variable can count the elements that satisfy a condition while the whole matrix is traversed.

**5. Algorithm / Procedure:**
1. Create the 3 by 3 matrix with the given values.
2. For first/last row: print the elements of row 0 and row 2 with a loop.
3. For column sums: outer loop over columns, inner loop over rows, add m[i][j] to the sum.
4. For counting: traverse all elements and increase the counter when the element is greater than 5.
5. Print the results.

**6. Program / Queries / Commands:**

**Task 1: First row and last row**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 9.1

public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        System.out.print("First row:");
        for (int j = 0; j < 3; j++) {
            System.out.print(" " + m[0][j]);
        }
        System.out.println();
        System.out.print("Last row:");
        for (int j = 0; j < 3; j++) {
            System.out.print(" " + m[2][j]);
        }
        System.out.println();
    }
}
```

**Task 2: Sum of each column**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 9.2

public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        for (int j = 0; j < 3; j++) {
            int sum = 0;
            for (int i = 0; i < 3; i++) {
                sum = sum + m[i][j];
            }
            System.out.println("Column " + (j + 1) + " sum: " + sum);
        }
    }
}
```

**Task 3: Count values greater than 5**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 9.3

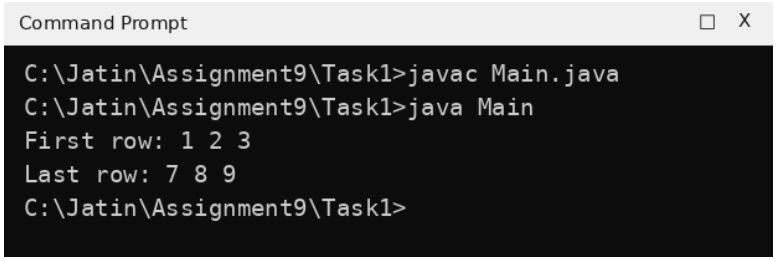
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int count = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (m[i][j] > 5) {
                    count++;
                }
            }
        }
        System.out.println("Values greater than 5: " + count);
    }
}
```

**7. Input:**

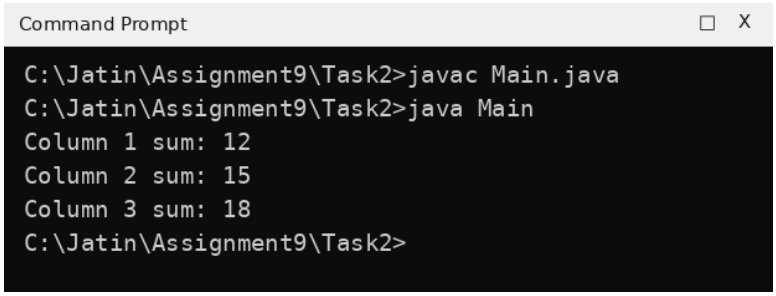
**Task 1:** Matrix rows: 1 2 3, 4 5 6, 7 8 9
**Task 2:** Matrix rows: 1 2 3, 4 5 6, 7 8 9
**Task 3:** Matrix rows: 1 2 3, 4 5 6, 7 8 9

**8. Output:**

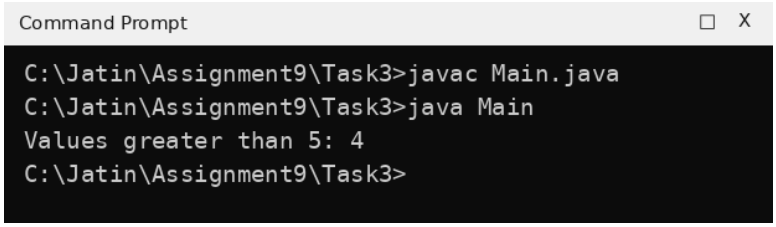
**Task 1: First row and last row**



**Task 2: Sum of each column**




**Task 3: Count values greater than 5**




**9. Result:**

The first and last rows were printed, column sums were calculated and values greater than 5 were counted correctly using nested loops.

# EXPERIMENT NO. 10

**1. Experiment Title:**
Switch and ArrayList

**2. Aim:**
To choose a branch with a switch statement, and to store and change a list of numbers in an ArrayList.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
The switch statement selects one block of code from many cases; break ends a case and default runs when no case matches. An ArrayList (java.util) is a resizable array. Important methods are add() to insert a value, get(index) to read a value, set(index, value) to change a value, remove(index) to delete a value, and size() to get the number of elements.

**5. Algorithm / Procedure:**
1. Import java.util.ArrayList when an ArrayList is used.
2. For switch: store the day number, write one case for each day and a default case.
3. For ArrayList: create the list, add the values and use get(), set(), remove() and size() as required.
4. Print the list and the other results.

**6. Program / Queries / Commands:**

**Task 1: Day name using switch**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 10.1

public class Main {
    public static void main(String[] args) {
        int day = 3;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

**Task 2: ArrayList: add, size and get**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 10.2
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);
        System.out.println(list);
        System.out.println("Size: " + list.size());
        System.out.println("Item at index 2: " + list.get(2));
    }
}
```

**Task 3: ArrayList: set and remove**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 10.3
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);
        list.set(1, 25);
        list.remove(3);
        System.out.println(list);
        System.out.println("Size: " + list.size());
    }
}
```

**7. Input:**

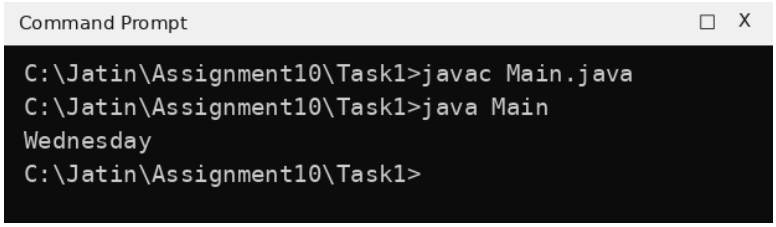
**Task 1:** day = 3

**Task 2:** Values: 10, 20, 30, 40

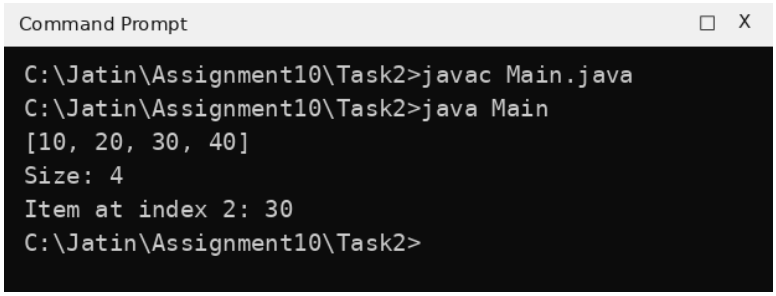
**Task 3:** Values: 10, 20, 30, 40; set index 1 to 25; remove index 3

**8. Output:**

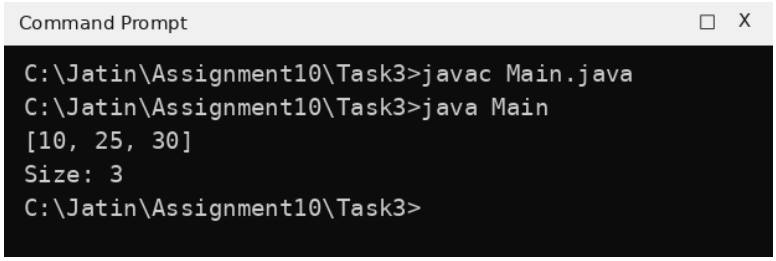
**Task 1: Day name using switch**



**Task 2: ArrayList: add, size and get**




**Task 3: ArrayList: set and remove**




**9. Result:**

The switch statement and the ArrayList methods add, get, set, remove and size were used successfully and the output was correct.

# EXPERIMENT NO. 11

**1. Experiment Title:**
Bubble Sort and Selection Sort

**2. Aim:**
To sort small arrays with bubble sort and selection sort, and count the swaps.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
Bubble sort repeatedly compares neighbouring elements and swaps them if they are in the wrong order, so the largest value moves to the end in every pass. Selection sort finds the smallest element of the unsorted part and places it at the front of that part. Both algorithms take O(n^2) time. Arrays.toString() converts an array to the text form [a, b, c].

**5. Algorithm / Procedure:**
1. Create the array with the given values.
2. Bubble sort: repeat n-1 passes; in each pass compare arr[j] and arr[j+1] and swap if arr[j] > arr[j+1] (count each swap).
3. Selection sort: for each position i find the index of the smallest element from i to the end and swap it with arr[i].
4. Print the sorted array (and the swap count).

**6. Program / Queries / Commands:**

**Task 1: Bubble sort**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 11.1
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

**Task 2: Selection sort**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 11.2
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};
        for (int i = 0; i < arr.length - 1; i++) {
            int min = i;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[min]) {
                    min = j;
                }
            }
            int temp = arr[min];
            arr[min] = arr[i];
            arr[i] = temp;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

**Task 3: Count swaps in bubble sort**

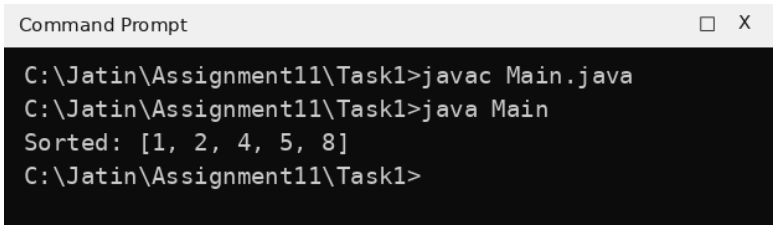
```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 11.3

public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        int swaps = 0;
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swaps++;
                }
            }
        }
        System.out.println("Swaps: " + swaps);
    }
}
```

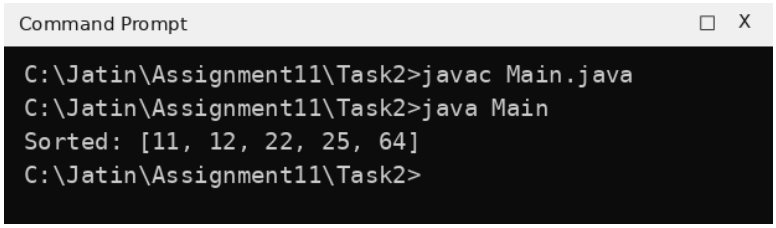
**7. Input:**
**Task 1:** arr = {5, 1, 4, 2, 8}
**Task 2:** arr = {64, 25, 12, 22, 11}
**Task 3:** arr = {5, 1, 4, 2, 8}

## 8. Output:

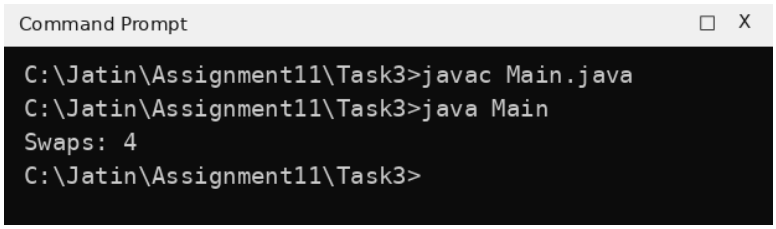
### Task 1: Bubble sort




### Task 2: Selection sort




### Task 3: Count swaps in bubble sort




## 9. Result:

Bubble sort and selection sort were implemented, the arrays were sorted in increasing order and the number of swaps (4) was counted correctly.

# EXPERIMENT NO. 12

**1. Experiment Title:**
Insertion Sort

**2. Aim:**
To sort small arrays with insertion sort and count how many values are shifted.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
Insertion sort builds the sorted part of the array one element at a time. Each new element (the key) is compared with the elements on its left; larger elements are shifted one place to the right and the key is inserted in the gap. Every move of one value one place to the right is called a shift. In the worst case (reverse order) it takes O(n^2) time.

**5. Algorithm / Procedure:**
1. Create the array with the given values.
2. For i = 1 to n-1: store key = arr[i] and set j = i - 1.
3. While j >= 0 and arr[j] > key: shift arr[j] to arr[j+1] (count the shift) and decrease j.
4. Place the key at arr[j+1].
5. Print the sorted array (and the number of shifts).

**6. Program / Queries / Commands:**

**Task 1: Insertion sort (7, 3, 5, 1)**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 12.1
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {7, 3, 5, 1};
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

**Task 2: Insertion sort (14, 3, 9, 21, 6)**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 12.2
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] arr = {14, 3, 9, 21, 6};
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

**Task 3: Count shifts**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 12.3

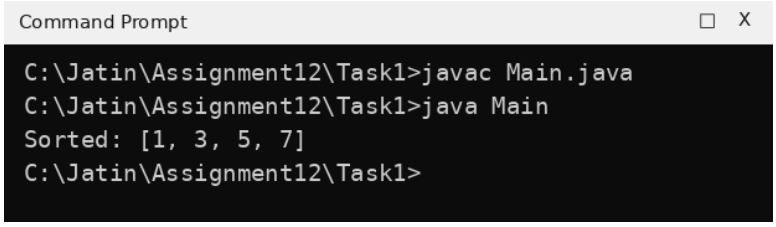
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 5, 4, 3, 2, 1};
        int shifts = 0;
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
                shifts++;
            }
            arr[j + 1] = key;
        }
        System.out.println("Shifts: " + shifts);
    }
}
```

**7. Input:**

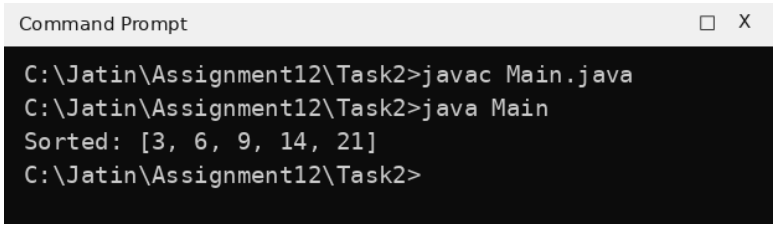
**Task 1:** arr = {7, 3, 5, 1}
**Task 2:** arr = {14, 3, 9, 21, 6}
**Task 3:** arr = {6, 5, 4, 3, 2, 1}

**8. Output:**

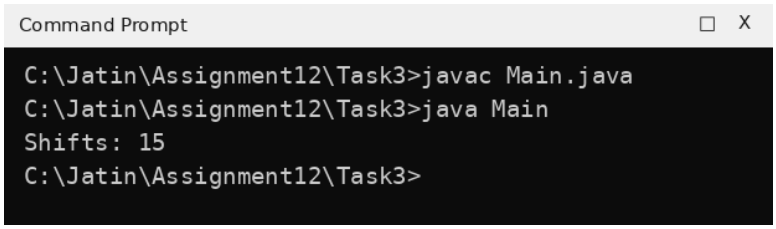
**Task 1: Insertion sort (7, 3, 5, 1)**



**Task 2: Insertion sort (14, 3, 9, 21, 6)**




**Task 3: Count shifts**




**9. Result:**

Insertion sort was implemented successfully, the arrays were sorted in increasing order and 15 shifts were counted for the reverse-ordered array.

# EXPERIMENT NO. 13

**1. Experiment Title:**
Strings: Methods and Palindromes

**2. Aim:**
To use common String methods, reverse a word, check a palindrome and print substrings.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
A String is a sequence of characters. Useful methods are length(), indexOf(), toUpperCase(), charAt(index) and substring(start, end). Strings are immutable, so a reversed word is built by joining characters with +. A palindrome reads the same from both ends, for example racecar. substring(i, j) returns the characters from index i up to j - 1.

**5. Algorithm / Procedure:**
1. Store the given word in a String variable.
2. Call the required String methods, or use a loop with charAt().
3. For palindrome: compare the reversed word with the original using equals().
4. For substrings: outer loop for the start index, inner loop for the end index and print substring(i, j).
5. Print the results.

**6. Program / Queries / Commands:**

**Task 1: String methods**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 13.1

public class Main {
    public static void main(String[] args) {
        String word = "Programming";
        System.out.println("Length: " + word.length());
        System.out.println("Position of g: " + word.indexOf('g'));
        System.out.println("Upper case: " + word.toUpperCase());
    }
}
```

**Task 2: Reverse a word**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 13.2

public class Main {
    public static void main(String[] args) {
        String word = "Java";
        String reversed = "";
        for (int i = word.length() - 1; i >= 0; i--) {
            reversed = reversed + word.charAt(i);
        }
        System.out.println("Reversed: " + reversed);
    }
}
```

**Task 3: Palindrome check**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 13.3

public class Main {
    public static void main(String[] args) {
        String word = "racecar";
        String reversed = "";
        for (int i = word.length() - 1; i >= 0; i--) {
            reversed = reversed + word.charAt(i);
        }
        if (word.equals(reversed)) {
            System.out.println(word + " is a palindrome");
        } else {
            System.out.println(word + " is not a palindrome");
        }
    }
}
```

**Task 4: All substrings**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 13.4

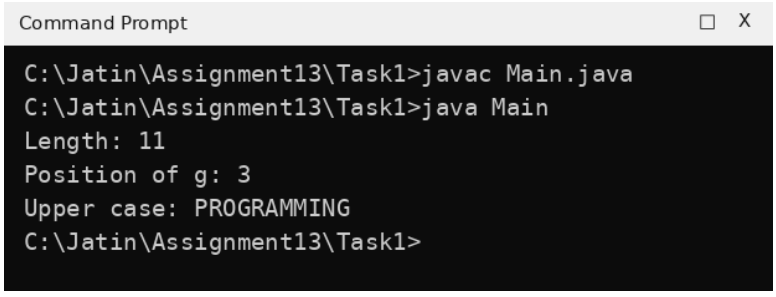
public class Main {
    public static void main(String[] args) {
        String word = "abc";
        for (int i = 0; i < word.length(); i++) {
            for (int j = i + 1; j <= word.length(); j++) {
                System.out.println(word.substring(i, j));
            }
        }
    }
}
```

**7. Input:**

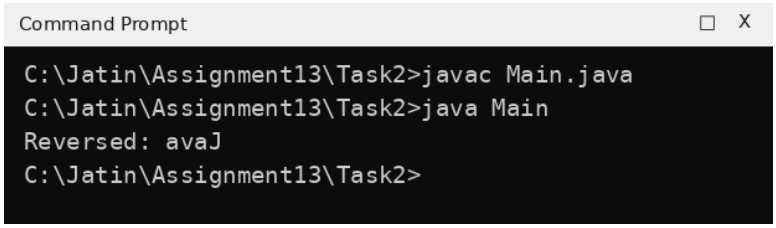
**Task 1:** word = "Programming"
**Task 2:** word = "Java"
**Task 3:** word = "racecar"
**Task 4:** word = "abc"

**8. Output:**

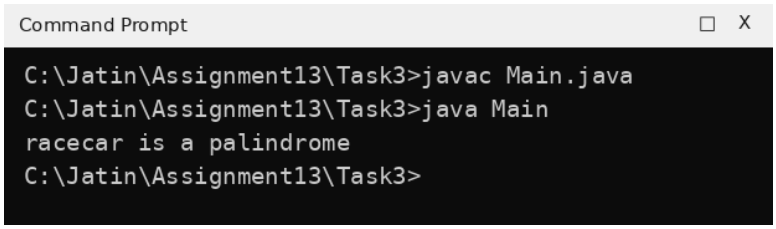
**Task 1: String methods**



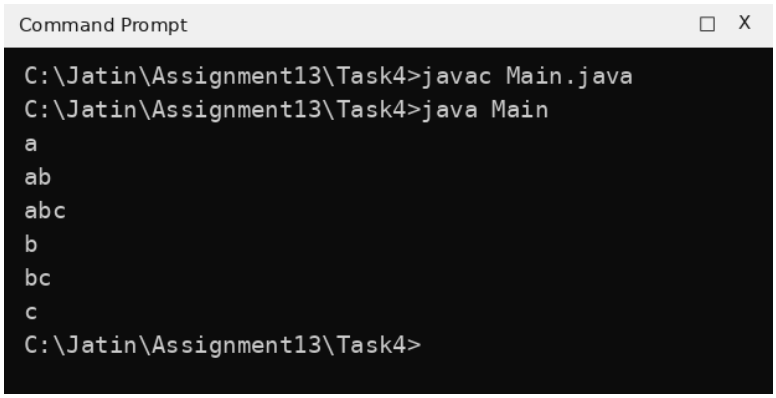
**Task 2: Reverse a word**




**Task 3: Palindrome check**




**Task 4: All substrings**




**9. Result:**

String methods, string reversal, palindrome checking and substring printing were done successfully and the output was correct.

# EXPERIMENT NO. 14

**1. Experiment Title:**
Characters: Counting and Positions

**2. Aim:**
To use charAt() to count vowels and a letter, and print each letter with its position.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
The charAt(index) method returns the character at a given position of a String; positions start from 0. A for loop from 0 to s.length() - 1 can visit every character. A counter variable is increased whenever the character matches a condition, such as being a vowel (a, e, i, o, u) or equal to a given letter.

**5. Algorithm / Procedure:**
1. Store the given text in the String s.
2. Run a loop from 0 to s.length() - 1 and read each character using charAt(i).
3. Compare the character with the vowels or the given letter and increase the counter, or print the position and the letter.
4. Print the final result.

**6. Program / Queries / Commands:**

**Task 1: Count vowels**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 14.1

public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                count++;
            }
        }
        System.out.println("Vowels: " + count);
    }
}
```

**Task 2: Count the letter g**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 14.2

public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == 'g') {
                count++;
            }
        }
        System.out.println("Count of g: " + count);
    }
}
```

**Task 3: Letters with positions**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 14.3

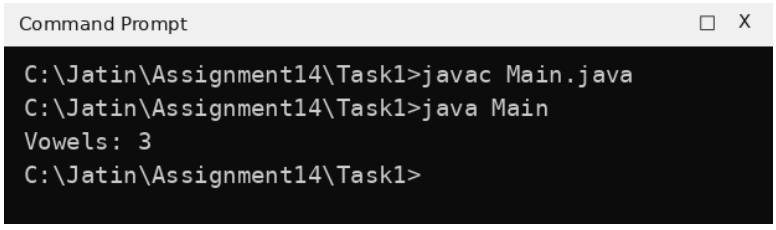
public class Main {
    public static void main(String[] args) {
        String s = "hello";
        for (int i = 0; i < s.length(); i++) {
            System.out.println(i + ": " + s.charAt(i));
        }
    }
}
```

**7. Input:**

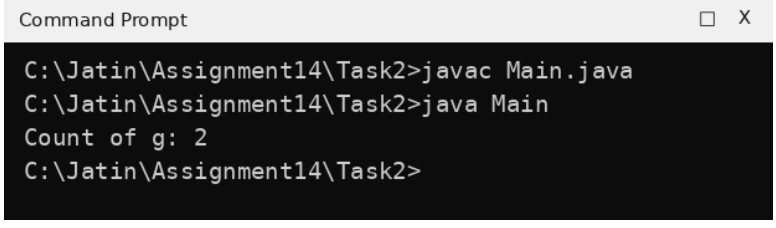
**Task 1:** s = "programming"
**Task 2:** s = "programming"
**Task 3:** s = "hello"

**8. Output:**

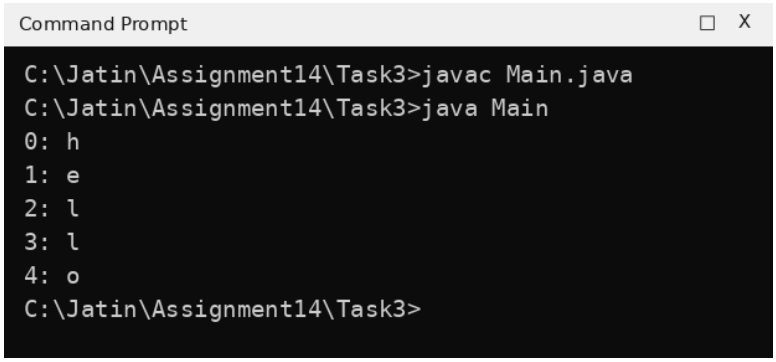
**Task 1: Count vowels**




**Task 2: Count the letter g**



**Task 3: Letters with positions**




**9. Result:**

Vowels and the letter g were counted, and every letter was printed with its position using charAt(). The output matched the expected output.

# EXPERIMENT NO. 15

**1. Experiment Title:**
StringBuilder Methods

**2. Aim:**
To build and change text with StringBuilder, using append, insert and deleteCharAt.

**3. Software / Tools Required:**
Java (JDK 21), Command Prompt / Terminal, and a text editor or IDE (VS Code / Notepad++).

**4. Theory:**
A String cannot be changed after it is created, but a StringBuilder is a mutable sequence of characters, so it can be modified without creating a new object each time. append() adds text at the end, insert(index, text) adds text at a given position, deleteCharAt(index) removes one character, and length() gives the number of characters.

**5. Algorithm / Procedure:**
1. Create a StringBuilder object with the given starting text (or empty).
2. Use append() to add text at the end, or insert() to add text at an index.
3. Use deleteCharAt(sb.length() - 1) to remove the last character where required.
4. Print the StringBuilder and its length.

**6. Program / Queries / Commands:**

**Task 1: append() and length()**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 15.1

public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        System.out.println(sb);
        System.out.println("Length: " + sb.length());
    }
}
```

**Task 2: append() and deleteCharAt()**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 15.2

public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 5; i++) {
            sb.append(i);
            sb.append(",");
        }
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb);
    }
}
```

**Task 3: insert()**

```
// Name: Jatin Rathor
// Roll No: BCS2026462
// Assignment: 15.3

public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.insert(0, "Hello ");
        System.out.println(sb);
    }
}
```

**7. Input:**

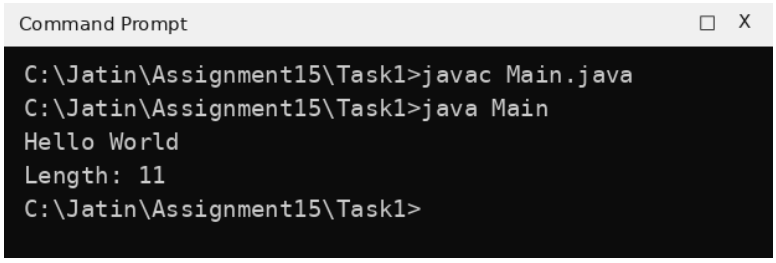
**Task 1:** Starting text = "Hello"; append " World"

**Task 2:** Numbers 1 to 5

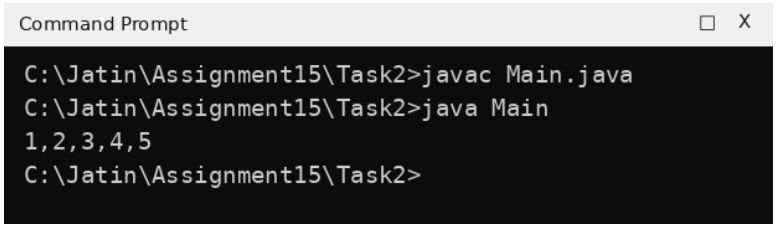
**Task 3:** Starting text = "Java"; insert "Hello " at index 0

**8. Output:**

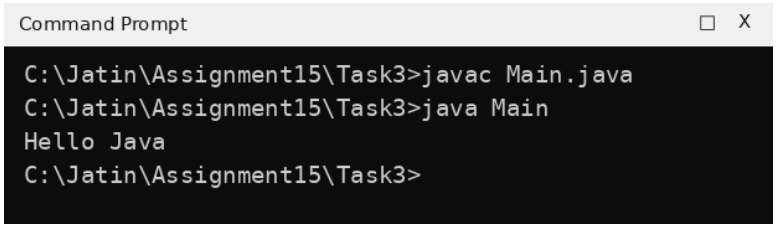
**Task 1: append() and length()**




**Task 2: append() and deleteCharAt()**




**Task 3: insert()**




**9. Result:**

append(), insert(), deleteCharAt() and length() of StringBuilder were used successfully and the output matched the expected output.